



Machine learning in finance

Supervised learning 3: advanced models

Prof. Dr. Branka Hadji Misheva

Prof. Dr. Lucía Gómez Teijeiro

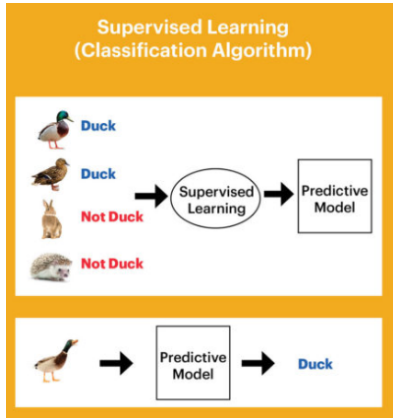
Prof. Dr. Ulrich Matter



Recap

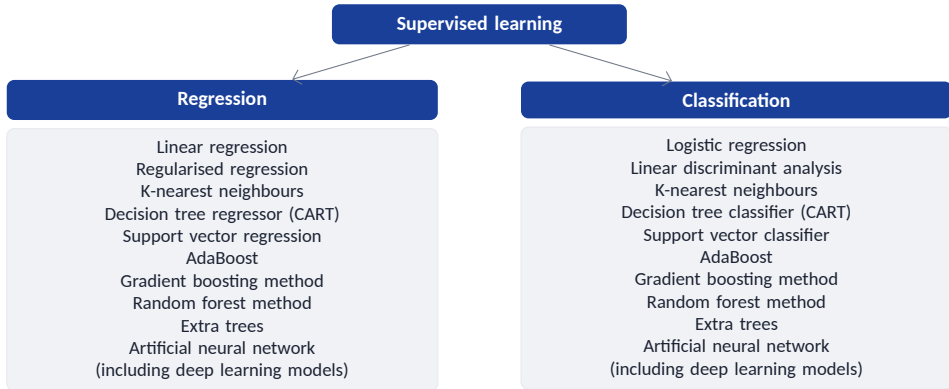


Supervised learning



We have something specific that we want to predict – or the available data consists of **labelled examples**

Supervised learning





Linear Regression



Continuous Y
Fits a straight line to data
Simple

Logistic Regression

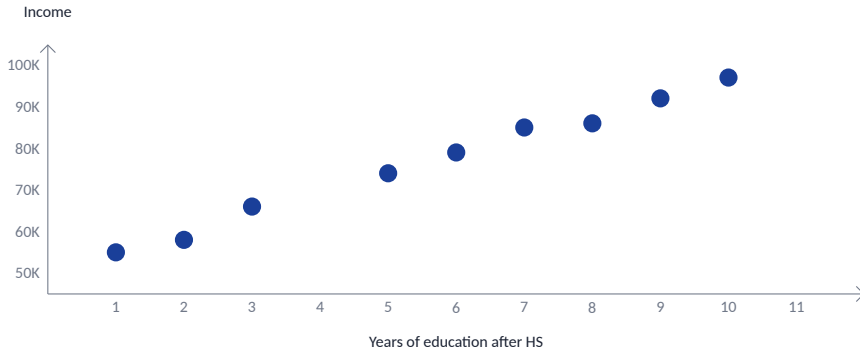
Decision Tree

Random Forest

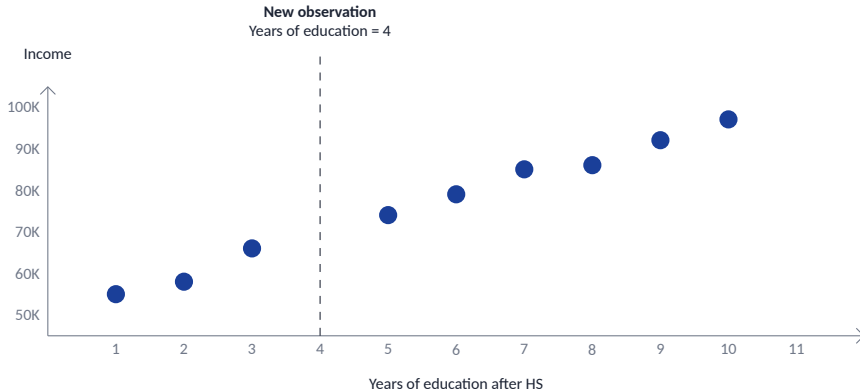
Gradient Boosting

Support Vector Machine

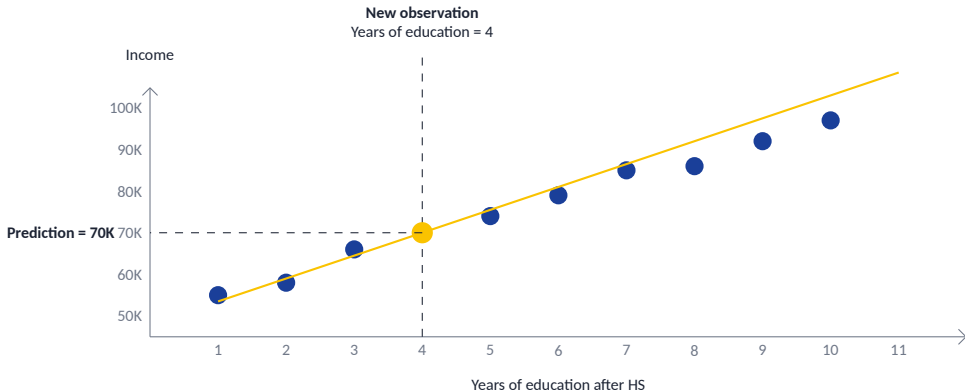
Linear regression



Linear regression



Linear regression



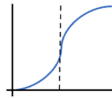


Linear Regression



Continuous Y
Fits a straight line to data
Simple

Logistic Regression



Categorical Y
Fits an S-shaped curve to data
Simple

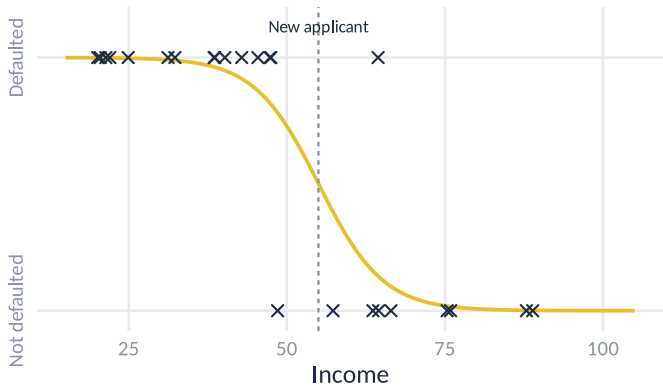
Decision Tree

Random Forest

Gradient Boosting

Support Vector Machine

When we have a discrete response!



Classification task

Should a loan application get approved?

Estimate a **probability** that an instance belongs to a particular **class**

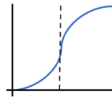


Linear Regression



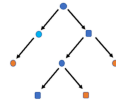
Continuous Y
Fits a straight line to data
Simple

Logistic Regression



Categorical Y
Fits an S-shaped curve to data
Simple

Decision Tree



Categorical or continuous Y
Splits data into rule-based branches
Interpretable

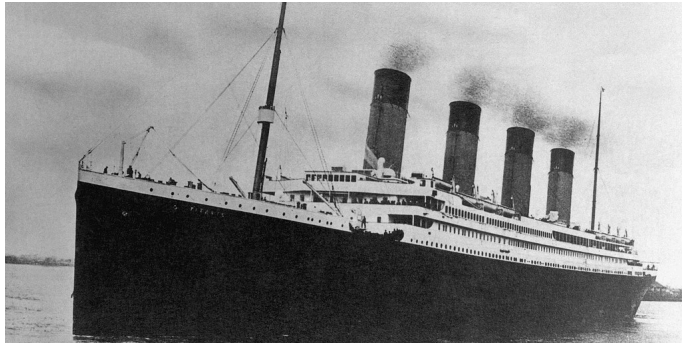
Random Forest

Gradient Boosting

Support Vector Machine

Let's think of an example ...

Let's think of the **Titanic**!

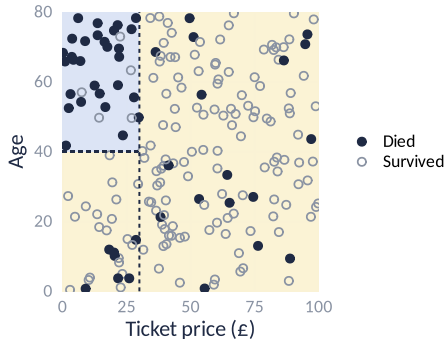


Let's think of an example ...

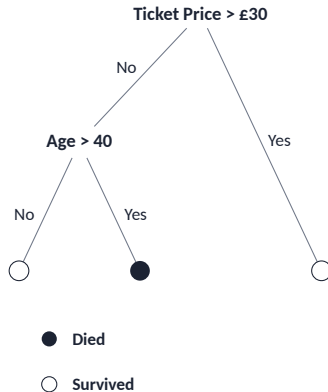


Could Jack have survived? Was there space on the door?

What about non-linear relationships?



Scatterplot of the people on board based on the ticket price they paid and their age.



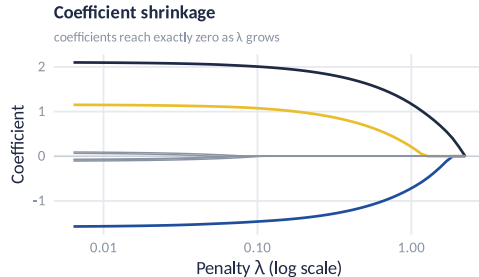
LASSO regression: introduction

LASSO stands for Least Absolute Shrinkage and Selection Operator

Used for both linear and logistic regression

Helps prevent overfitting and performs feature selection automatically

Adds a penalty on large coefficients





LASSO in linear regression

Extends linear regression with an L1 penalty term

Objective: minimise

$$\text{RSS} + \lambda \sum_i |\beta_i|$$

Coefficients can become exactly zero $\rightarrow$ variable selection

$\lambda = 0$ gives ordinary least squares; as λ grows the coefficients shrink towards zero one after another (the paths on the previous slide) until only the intercept is left

Helps when you have many predictors, few of which matter (λ chosen by cross-validation; standardise features first)



LASSO in logistic regression

Works similarly to linear LASSO but for classification tasks

Penalises the logistic loss with an L1 penalty

Selects the most important predictors for distinguishing between classes

Common in credit scoring, fraud detection, and risk modelling



LASSO vs Ridge vs Elastic Net

01

Ridge

L2 (squared) penalty → shrinks coefficients but keeps all variables

02

LASSO

L1 (absolute) penalty → shrinks and removes some coefficients

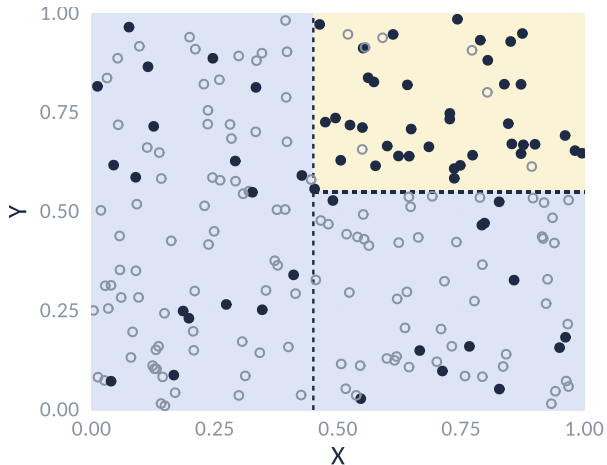
03

Elastic Net

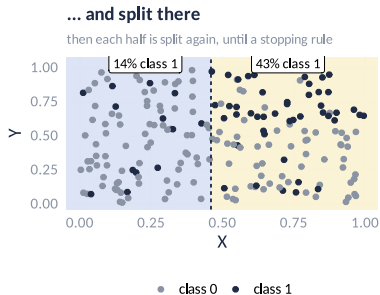
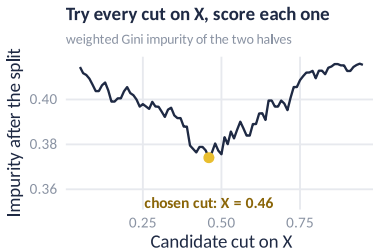
Combination of L1 and L2 penalties → balanced approach

LASSO is best when only a few predictors are truly important

Decision trees



How a tree chooses a **split**



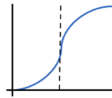
Greedy, one cut at a time: at each node the tree tries every variable and every cut, scores each by how pure the two halves become (Gini impurity for classes, squared error for a numeric Y), and keeps the best; it stops at a minimum node size or depth. Grown without a stopping rule, a tree memorises the training data – the reason forests and boosting exist.

Linear Regression



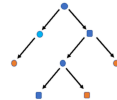
Continuous Y
Fits a straight line to data
Simple

Logistic Regression



Categorical Y
Fits an S-shaped curve to data
Simple

Decision Tree



Categorical or continuous Y
Splits data into rule-based branches
Interpretable

Random Forest



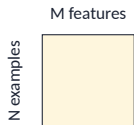
Categorical or continuous Y
Combines many trees for better predictions
Reduces overfitting

Gradient Boosting

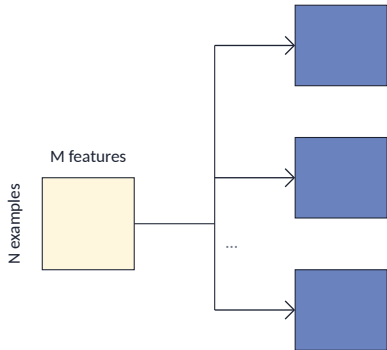
Support Vector Machine

Random forest

We start with the full data

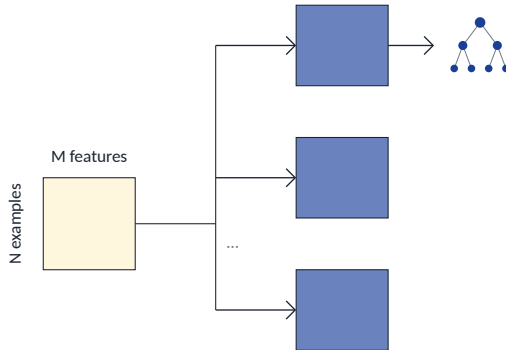


Random forest



Next, we create bootstrap samples from the training data

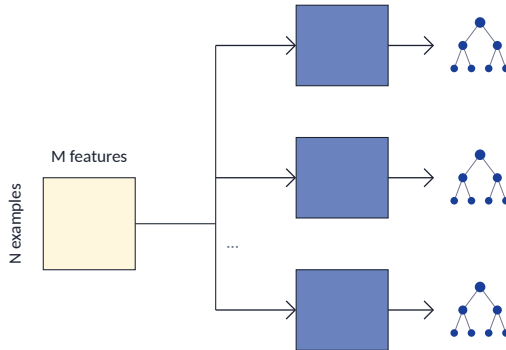
Random forest



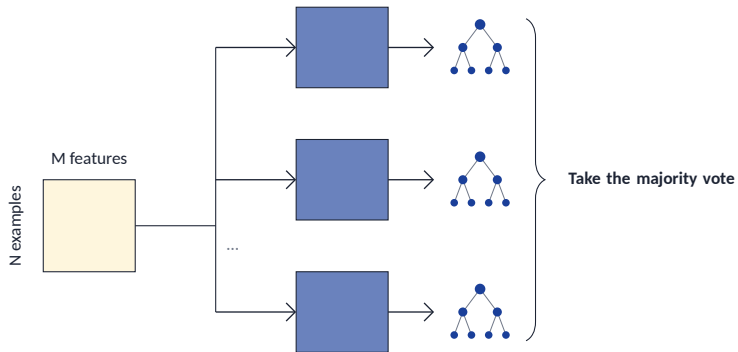
Next, from each subset we construct a decision tree

Random forest

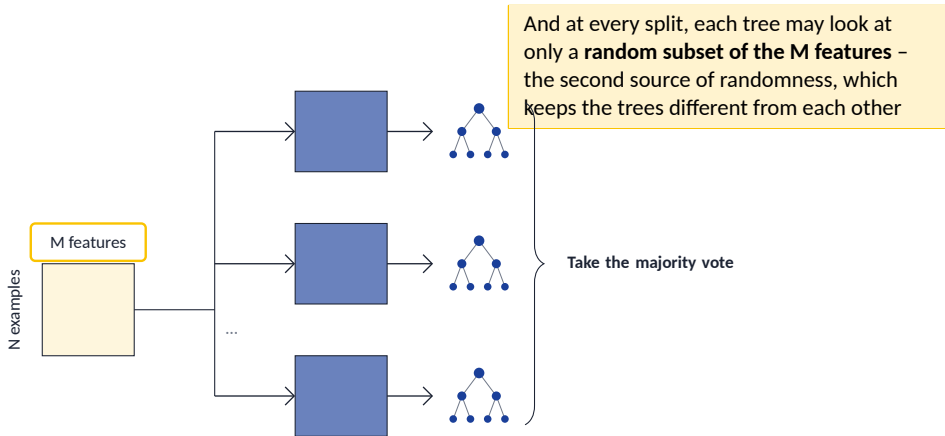
We do the same for each sample



Random forest



Random forest

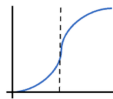


Linear Regression



Continuous Y
Fits a straight line to data
Simple

Logistic Regression



Categorical Y
Fits an S-shaped curve to data
Simple

Decision Tree



Categorical or continuous Y
Splits data into rule-based branches
Interpretable

Random Forest



Categorical or continuous Y
Combines many trees for better predictions
Reduces overfitting

Gradient Boosting

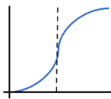
Support Vector Machine

Linear Regression



Continuous Y
Fits a straight line to data
Simple

Logistic Regression



Categorical Y
Fits an S-shaped curve to data
Simple

Decision Tree



Categorical or continuous Y
Splits data into rule-based branches
Interpretable

Random Forest



Categorical or continuous Y
Combines many trees for better predictions
Reduces overfitting

Gradient Boosting



Categorical or continuous Y
Adds trees that fix the previous ones' errors
Very accurate

Support Vector Machine



Gradient boosting

Gradient Boosting is an ensemble learning technique that builds a strong predictive model by sequentially improving weak models (typically decision trees)

Each new model is trained to correct the errors of the previous ones, reducing bias over iterations

It optimises predictions by minimising a loss function using gradient descent

The final model is a weighted sum of all the weak models, resulting in high accuracy and robustness

Source: displayr.com

Gradient boosting: intuition

What is it?

A smart way to improve predictions by fixing mistakes step by step.

How does it work?



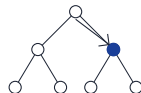
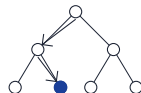
Step 1. Start with a simple model (weak learner).

Step 2. Find where it makes mistakes.

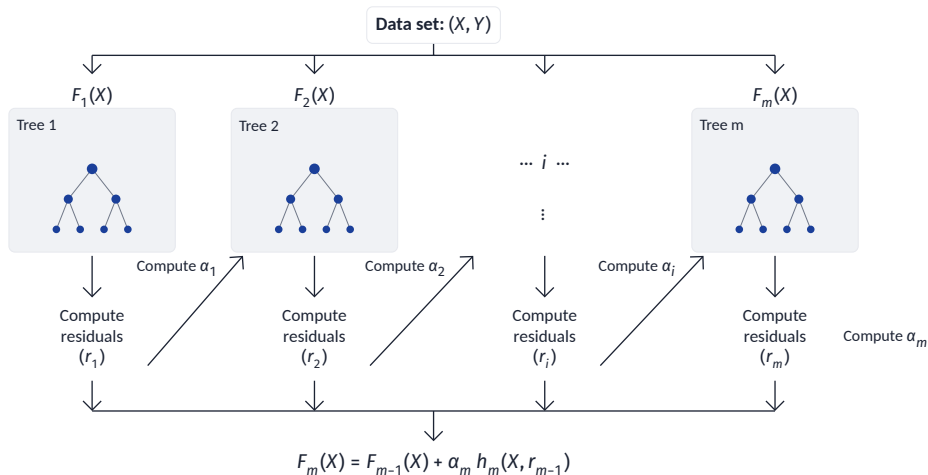
Step 3. Train a new model to correct those mistakes.

Step 4. Repeat the process multiple times.

Step 5. Combine all models to make a strong final prediction.



Gradient boosting: the sequence of trees



$$F_m(X) = F_{m-1}(X) + \alpha_m h_m(X)$$

where h_m is the m th tree, trained to predict the residuals $r_{m-1,i} = Y_i - F_{m-1}(X_i)$ of the model so far from X , and α_m is its step weight. The weight is chosen to reduce the loss over all n observations:

$$\alpha_m = \arg \min_{\alpha} \sum_{i=1}^n L(Y_i, F_{m-1}(X_i) + \alpha h_m(X_i))$$

where $L(Y, F(X))$ is a differentiable loss function (for squared-error loss the residuals are exactly the negative gradient of L with respect to the predictions).

By training the second model on the gradient of the loss with respect to the predictions of the first model, it has learned to correct the mistakes of the first model.



Random forest vs gradient boosting

How are the weak learners built?

Unlike random forests, the decision trees in gradient boosting are built additively; in other words, each decision tree is built one after another – each new tree is fitted to the residuals of the trees before it, whereas the trees of a random forest are grown independently on bootstrap samples.

How are results aggregated?

In random forests, the results of decision trees are aggregated at the end of the process (the average, or the majority vote). Gradient boosting does not do this and instead aggregates the results of each decision tree along the way to calculate the final result: the prediction is the sum of all trees, each scaled by its step weight.



Linear regression vs gradient boosting

Linear Regression

Both aim to find the best way to predict an output (Y) given an input (X)

Assumes that the relationship between the input (X) and the output (Y) follows a **fixed linear function**:

$$Y = \beta X + \alpha$$

where the α and β are the **parameters** that the model learn

The goal is to **find the best values** for α and β that minimise the sum of squared errors.

It does **NOT** change the function (it always remains linear).

Gradient Boosting

Gradient boosting does not assume a fixed function form.

Instead, it **searches for the best function** by iteratively adding small models (often decision trees) to correct the previous model's mistakes.

Step 1. Start with a simple model: $Y^{(0)} = \bar{Y}$

Step 2. Calculate the residuals: $r_i = Y_i - Y^{(0)}$

Step 3. Search for the best function (and best values for its parameters) to reduce errors:

$$h_1(X) = \begin{cases} y_1 & \text{if } X < x_1 \\ y_2 & \text{if } X \geq x_2 \end{cases}$$



Linear regression vs gradient boosting

Linear Regression

Both aim to find the best way to predict an output (Y) given an input (X)

Assumes that the relationship between the input (X) and the output (Y) follows a **fixed linear function**:

$$Y = \beta X + \alpha$$

where the α and β are the **parameters** that the model learn

The goal is to **find the best values** for α and β that minimise the sum of squared errors.

It does **NOT** change the function (it always remains linear).

Gradient Boosting

Gradient boosting does not assume a fixed function form.

Instead, it **searches for the best function** by iteratively adding small models (often decision trees) to correct the previous model's mistakes.

Step 1. Start with a simple model: $Y^{(0)} = \bar{Y}$

Step 2. Calculate the residuals: $r_i = Y_i - Y^{(0)}$

Step 3. Search for the best function (and best values for its parameters) to reduce errors:

$$h_1(X) = \begin{cases} y_1 & \text{if } X_1 < x_1 \text{ and } X_2 < x_2 \\ y_2 & \text{if } X_1 < x_1 \text{ and } X_2 > x_2 \\ y_3 & \text{if } X_1 \geq x_1 \end{cases}$$

Linear regression vs gradient boosting

Linear Regression

Both aim to find the best way to predict an output (Y) given an input (X)

Assumes that the relationship between the input (X) and the output (Y) follows a **fixed linear function**:

$$Y = \beta X + \alpha$$

where the α and β are the **parameters** that the model learn

The goal is to **find the best values** for α and β that minimise the sum of squared errors.

It does **NOT** change the function (it always remains linear).

Gradient Boosting

Gradient boosting does not assume a fixed function form.

Instead, it **searches for the best function** by iteratively adding small models (often decision trees) to correct the previous model's mistakes.

Step 1. Start with a simple model: $Y^{(0)} = \bar{Y}$

Step 2. Calculate the residuals: $r_i = Y_i - Y^{(0)}$

Step 3. Search for the best function (and best values for its parameters) to reduce errors:

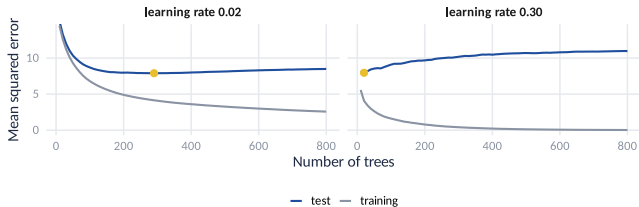
$$h_1(X) = \begin{cases} y_1 & \text{if } X_1 < x_1 \text{ and } X_2 < x_2 \\ y_2 & \text{if } X_1 < x_1 \text{ and } X_2 > x_2 \\ y_3 & \text{if } X_1 \geq x_1 \end{cases}$$

As you might imagine, this makes it quite slow ... and boosting **can overfit** (next slides)

Boosting can **overfit**: the controls

Boosting keeps fitting: training error falls, test error turns

gold dot: the number of trees with the lowest test error (early stopping)



Number of trees: training error keeps falling, test error turns up – stop where it is lowest (**early stopping**)

Learning rate: scale each tree down; smaller steps need more trees but overfit later and more gently

Depth and **subsampling** limit what one tree can memorise

A random forest does not overfit with more trees: averaging independent trees only reduces variance

XGBoost

eXtreme Gradient Boosting

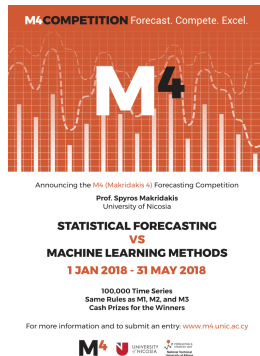
One of the fastest implementations of gradient boosting trees

XGBoost is an incredibly popular machine learning library

It was developed originally as a C++ command-line application

After performing well in several popular machine learning competitions, the package started being adopted within the ML community

As a result, now we have available implementations in many programming languages including **Python, R, Scala, and Julia**



M4 COMPETITION Forecast. Compete. Excel.

M4

Announcing the **M4 (Makridakis 4)** Forecasting Competition
Prof. Spyros Makridakis
University of Nicosia

STATISTICAL FORECASTING
VS
MACHINE LEARNING METHODS
1 JAN 2018 - 31 MAY 2018

100,000 Time Series
Same Rules as M1, M2, and M3
Cash Prizes for the Winners

For more information and to submit an entry: www.m4.unic.ac.cy

M4 UNIVERSITY OF NICOSIA UNIVERSITY OF Nicosia

Source: DataCamp, Extreme gradient boosting with XGBoost



XGBoost: advantages

Speed:

- it can harness all of the processing power of modern multi-core computers

- it is parallelisable onto GPUs and across networks of computers

- Hence: it is suited for training models on very large datasets (**hundreds of millions of training examples**)

Performance:

- It is among the strongest single-algorithm methods in ML competitions

- It achieves state-of-the-art performance on a variety of benchmark ML datasets



Putting all together until now ...

Let's try to summarise through **an example**

Imagine you are the manager of a company and you are deciding who, among a list of applicants, to hire
How would the decision happen under the different methods we discussed so far?

Putting all together

Single decision tree



The manager has a set of criteria such as skills acquired, education level, number of years in industry, etc., on which it scores the candidates

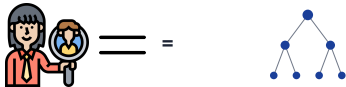
Random forest

Gradient boosting

XGBoost – enhanced gradient boosting

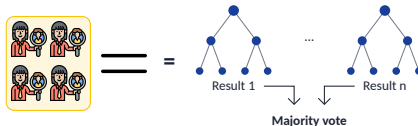
Putting all together

Single decision tree



The manager has a set of criteria such as skills acquired, education level, number of years in industry, etc., on which it scores the candidates

Random forest



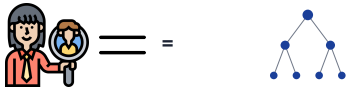
We now have a **panel** and each interviewer has a vote. Each interviewer will also score the applicants based on a **randomly selected subset of qualifications**.

Gradient boosting

XGBoost – enhanced gradient boosting

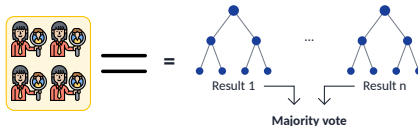
Putting all together

Single decision tree



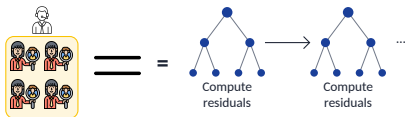
The manager has a set of criteria such as skills acquired, education level, number of years in industry, etc., on which it scores the candidates

Random forest



We now have a **panel** and each interviewer has a vote. Each interviewer will also score the applicants based on a **randomly selected subset of qualifications**.

Gradient boosting

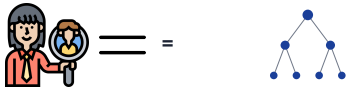


Each interviewer alters the evaluation criteria based on the feedback of previous interviewers.

XGBoost - enhanced gradient boosting

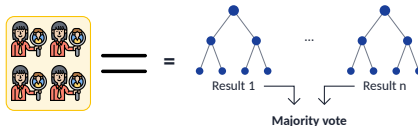
Putting all together

Single decision tree



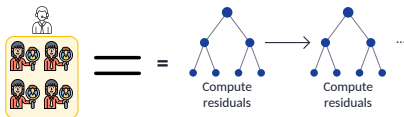
The manager has a set of criteria such as skills acquired, education level, number of years in industry, etc., on which it scores the candidates

Random forest



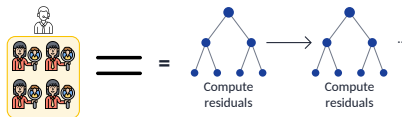
We now have a **panel** and each interviewer has a vote. Each interviewer will also score the applicants based on a **randomly selected subset of qualifications**.

Gradient boosting



Each interviewer alters the evaluation criteria based on the feedback of previous interviewers.

XGBoost - enhanced gradient boosting



As gradient boosting, plus **software and hardware optimisation techniques**

Let's go on ...

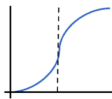


Linear Regression



Continuous Y
Fits a straight line to data
Simple

Logistic Regression



Categorical Y
Fits an S-shaped curve to data
Simple

Decision Tree



Categorical or continuous Y
Splits data into rule-based branches
Interpretable

Random Forest



Categorical or continuous Y
Combines many trees for better predictions
Reduces overfitting

Gradient Boosting



Categorical or continuous Y
Adds trees that fix the previous ones' errors
Very accurate

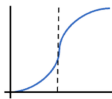
Support Vector Machine

Linear Regression



Continuous Y
Fits a straight line to data
Simple

Logistic Regression



Categorical Y
Fits an S-shaped curve to data
Simple

Decision Tree



Categorical or continuous Y
Splits data into rule-based branches
Interpretable

Random Forest



Categorical or continuous Y
Combines many trees for better predictions
Reduces overfitting

Gradient Boosting



Categorical or continuous Y
Adds trees that fix the previous ones' errors
Very accurate

Support Vector Machine



Categorical Y (regression variant)
Finds the boundary with the widest margin
Kernels for curved boundaries

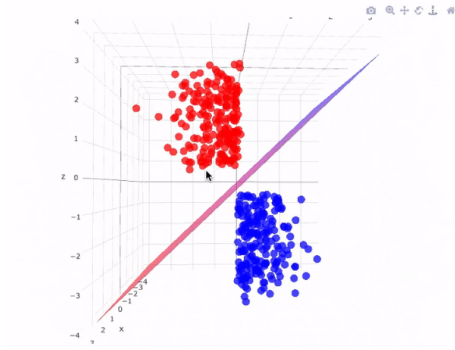
SVM

SVM → Support Vector Machine

Another type of supervised machine learning algorithm that is commonly used for **classification and regression**

SVMs try to find the hyperplane that best separates data into different classes, with the goal of maximising the margin, or distance, between the hyperplane and the closest data points from each class

Despite their effectiveness, **SVMs can be computationally intensive** and require careful tuning of their parameters



SVM: intuition

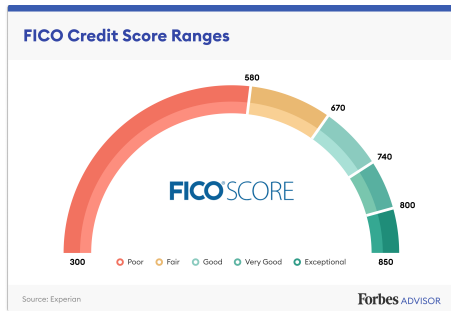
We start with a binary classification – problem that has two classes

Loan status: **default or non-default**

Credit score: this is a feature that ranges from 300 to 850. Applicants that score around 740 are considered of good credit score while applicants that score around 550 are considered of poor credit score

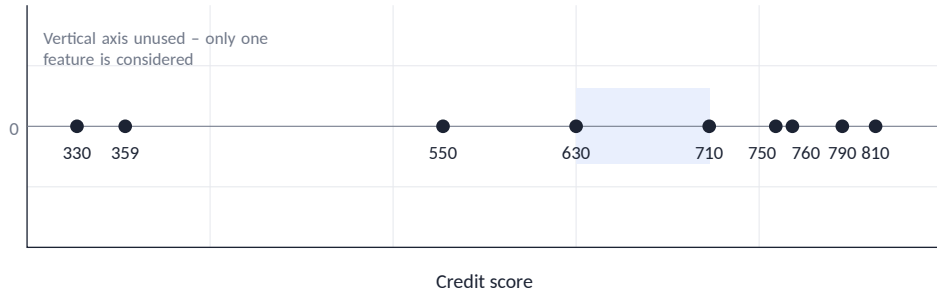
Of course, in practice, the credit score of good vs bad applicants (i.e. non-defaulted vs defaulted loans) **varies quite a bit**

Given these sample applicants, **our task is to determine a decision rule to distinguish between the two groups!**



A separating boundary between classes is called a **separator or decision boundary**

Any point in this range can be an acceptable separating boundary between the classes





Let's take two points in this area: **650** and **690**

An important concept is that of the **margin**, which is the distance between the decision boundary and the closest data point





For example, for the decision boundary **690**, the closest data point is **710** so **the margin is 20**



The best decision boundary is one **that maximises the margin!**





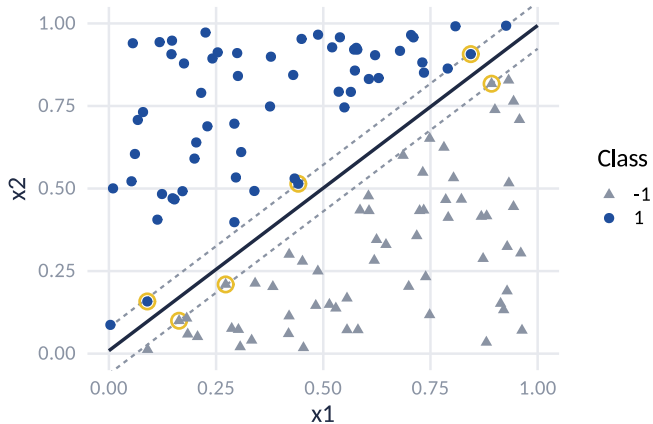
Looking at the graph, which is the best decision boundary?



The maximal margin separator lies halfway between the two clusters. That is, at the **midpoint of the line joining the sample data points**

This simple example serves to illustrate a key feature of SVM algorithms → **they find decision boundaries that maximise the margin**. In general: halfway between the closest points of the two classes, the **support vectors**





We choose the separator that gives the **maximal margin!**

How is the separator i.e., decision boundary **identified?**

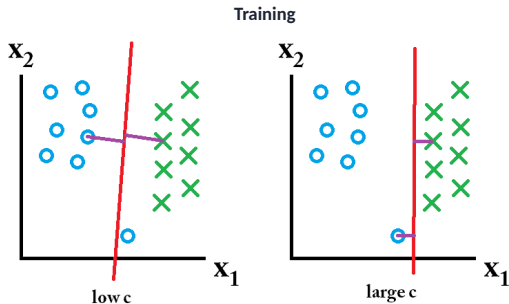
Note: C parameter tuning for SVM

In a SVM, you are searching for two things:

- (i) a hyperplane with the largest margin
- (ii) a hyperplane that correctly separates as many instances as possible

It might not be always possible to **get both things**

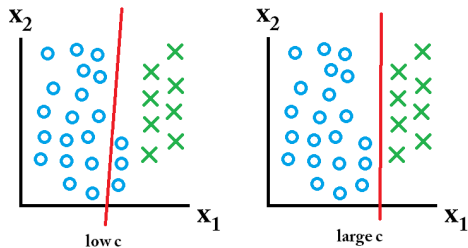
The C parameter determines how much you favour the second goal



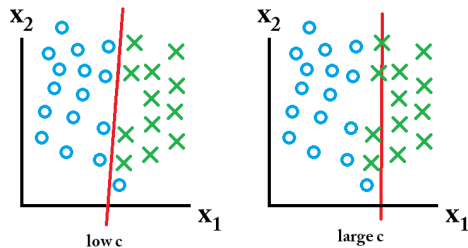
High C (Less Regularisation) →
Harder Margin, Fewer
Misclassifications on the training
data

Low C (More Regularisation) →
Softer Margin, More generalisation
– fewer training errors can mean
more errors on new data (next slide)

Note: C parameter tuning for SVM



Possible future data set 1



Possible future data set 2

Soft margin and the kernel idea

Soft margin: when the classes overlap, no boundary separates them. The SVM then lets some points sit inside the margin or on the wrong side, each at a cost, and maximises

margin - $C \times$ (total cost of the violations)

large C : violations are expensive - narrow margin, follows the training points

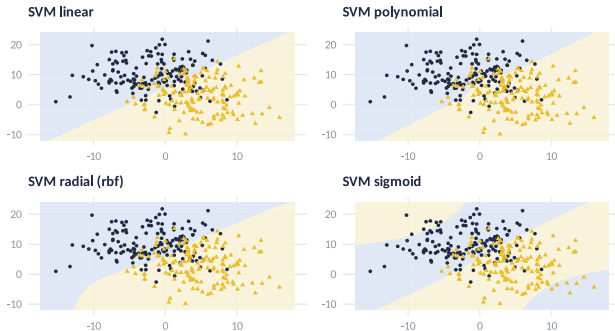
small C : violations are cheap - wide margin, smoother boundary, better on new data

Kernel idea: a boundary that is curved in the original variables can be a straight line in a richer space.



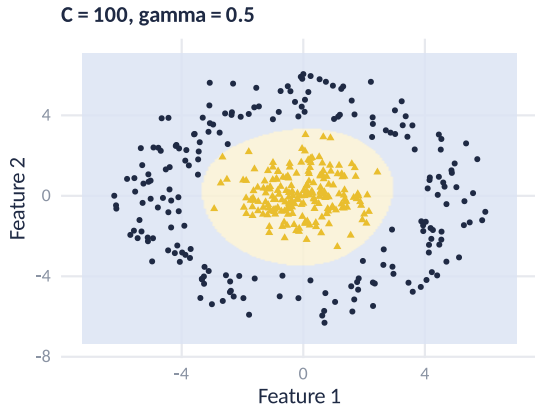
The kernel does this mapping implicitly

SVM: decision boundaries



The **kernel** is the similarity measure the SVM uses between two points: a linear kernel gives a straight boundary; polynomial and radial (RBF) kernels give curved ones, because they compare points as if mapped into a higher-dimensional space. It is chosen up front.

SVM: the radial basis function kernel



The RBF kernel measures similarity by distance: a point's influence fades with distance

So the boundary can close around a group of points – here a blob inside a ring, which no straight line separates

γ sets how far one point's influence reaches: large γ gives tight, wiggly boundaries, small γ smooth ones

C as before: the cost of a training error

Both are tuned by cross-validation; the picture uses $C = 100$, $\gamma = 0.5$

The models side by side

MODEL	Y	NON-LINEAR?	INTERPRETABLE?	OVERFITTING CONTROL	TYPICAL USE
Linear / logistic regression	continuous / class	no (unless you add terms)	high: one coefficient per variable	few parameters; LASSO/ridge	pricing, PD estimation, baselines
Decision tree	both	yes, axis-aligned	high while shallow	depth, minimum node size	rules for humans, a first look
Random forest	both	yes	low (feature importance only)	more trees never hurt; feature subsampling	robust default on tabular data
Gradient boosting / XGBoost	both	yes	low	trees, learning rate, depth, early stopping	best accuracy on tabular data, competitions
SVM	class (regression variant exists)	with a kernel	low	C, kernel, γ	medium-sized data, clear margins

Rule of thumb: start with the interpretable model; move to a forest or boosting when the extra accuracy is worth the loss of transparency – and always compare on held-out data.

Let's see it!





Notebooks

Advanced models (Python): https://colab.research.google.com/github/umatter/EDFB/blob/main/notebooks/Python/03_EAIF_Advanced_Models_Python.ipynb

Advanced models (R): https://colab.research.google.com/github/umatter/EDFB/blob/main/notebooks/R/03_EAIF_Advanced_Models_R.ipynb

Thank you

Questions and feedback are welcome any time.